

Performance Metrics For Regression Task

1. Mean Squared Error (MSE)

Explanation

Mean Squared Error (MSE) measures the average squared difference between estimated and actual values. It gives higher weight to larger errors due to squaring.

Mathematical Formula

$$MSE = (1/n) * \sum [(y_i - \hat{y}_i)^2]$$

Where:

- n is the number of data points.
- y_i is the actual value.
- $\hat{y}_i$ is the predicted value.

When to Use

- Sensitive to outliers
- Regression model performance evaluation
- Requires normally distributed errors
- Penalizes large errors more heavily

Python Implementation

```
from sklearn.metrics import mean_squared_error
import numpy as np

def calculate_mse(y_true, y_pred):
    return mean_squared_error(y_true, y_pred)
```

Pros

- Simple to understand
- Mathematically convenient

- Differentiable
- Symmetric error measurement

Cons

- Sensitive to outliers
- Uses squared values, making interpretation challenging
- Units are squared original units

Fun Facts

- Commonly used in machine learning optimization
- Fundamental in least squares regression
- Minimized by linear regression

2. Root Mean Squared Error (RMSE)

Explanation

RMSE is the square root of MSE, bringing the metric back to the original data scale.

Mathematical Formula

$$RMSE = \sqrt{[(1/n) * \sum[(y_i - \hat{y}_i)^2]]}$$

When to Use

- Need error in original units
- Want to penalize large errors
- Regression model comparison

Python Implementation

```
from sklearn.metrics import mean_squared_error
import numpy as np

def calculate_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))
```

Pros

- Interpretable in original units
- Sensitive to large errors
- Symmetric error measurement

Cons

- Sensitive to outliers
- Squares errors before taking root
- Less intuitive for non-technical audiences

Fun Facts

- Standard deviation of residuals
- Often preferred over MSE for reporting

3. Mean Absolute Error (MAE)

Explanation

MAE calculates the average absolute difference between predicted and actual values.

Mathematical Formula

$$MAE = (1/n) * \sum |y_i - \hat{y}_i|$$

When to Use

- Less sensitive to outliers
- Linear error measurement
- Robust regression evaluation

Python Implementation

```
from sklearn.metrics import mean_absolute_error

def calculate_mae(y_true, y_pred):
    return mean_absolute_error(y_true, y_pred)
```

Pros

- Less sensitive to outliers
- Interpretable
- Linear error measurement
- Uses absolute value

Cons

- Doesn't differentiate overestimation vs underestimation
- Less mathematically convenient for optimization
- Less penalization of large errors

Fun Facts

- Also known as L1 loss
- More robust for non-Gaussian error distributions

4. Mean Absolute Percentage Error (MAPE)

Explanation

MAPE represents the average percentage difference between predicted and actual values.

Mathematical Formula

$$MAPE = (1/n) * \sum |(y_i - \hat{y}_i)/y_i| * 100$$

When to Use

- Percentage-based error comparison
- Similar scale data
- Forecasting and time series

Python Implementation

```
import numpy as np

def calculate_mape(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

Pros

- Scale-independent
- Easy percentage interpretation
- Comparable across different scales

Cons

- Undefined when true value is zero
- Biased towards smaller values
- Asymmetric error treatment

Fun Facts

- Commonly used in financial forecasting
- Can be problematic with small true values

5. R-squared (Coefficient of Determination)

Explanation

R-squared represents the proportion of variance in the dependent variable predictable from independent variables.

Mathematical Formula

$$R^2 = 1 - (SS_{res} / SS_{tot})$$

Where:

- SS_{res} is the sum of squares of residuals
- SS_{tot} is the total sum of squares

When to Use

- Model goodness-of-fit assessment
- Linear regression evaluation
- Comparing model predictive power

Python Implementation

```
from sklearn.metrics import r2_score

def calculate_r2(y_true, y_pred):
    return r2_score(y_true, y_pred)
```

Pros

- Easy interpretation (0-1 range)
- Indicates model explanatory power
- Normalized measure

Cons

- Doesn't indicate model accuracy
- Can be misleading with non-linear relationships
- Increases with more predictors

Fun Facts

- Ranges from 0 to 1
- Not always best for model selection

6. Adjusted R-squared

Explanation

Adjusted R-squared penalizes adding unnecessary predictors to the model.

Mathematical Formula

$$AdjustedR^2 = 1 - [(1 - R^2) * (n - 1) / (n - p - 1)]$$

Where:

- n is the number of data points
- p is the number of predictors

When to Use

- Complex models with multiple predictors
- Preventing overfitting
- Model complexity comparison

Python Implementation

```
import numpy as np
from sklearn.linear_model import LinearRegression

def calculate_adjusted_r2(X, y):
    model = LinearRegression()
    model.fit(X, y)
    r2 = model.score(X, y)
    n, p = X.shape
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
    return adj_r2
```

Pros

- Prevents overfitting
- Accounts for model complexity
- More reliable for complex models

Cons

- Still has limitations
- Assumes linear relationship
- Not suitable for non-linear models

Fun Facts

- Used in feature selection

- Developed to improve R-squared limitations

7. Root Mean Squared Logarithmic Error (RMSLE)

Explanation

RMSLE calculates root mean squared error after log transformation, reducing impact of large errors.

Mathematical Formula

$$RMSLE = \sqrt{[(1/n) * \sum[(\log(p_i + 1) - \log(y_i + 1))^2]]}$$

When to Use

- Percentage errors matter more than absolute errors
- Skewed data
- Predicting exponential growth

Python Implementation

```
import numpy as np

def calculate_rmsle(y_true, y_pred):
    return np.sqrt(np.mean((np.log1p(y_pred) - np.log1p(y_true))**2))
```

Pros

- Less sensitive to outliers
- Handles exponential growth
- Logarithmic scale benefits

Cons

- Complex interpretation
- Less intuitive
- Requires log transformation

Fun Facts

- Popular in Kaggle competitions
 - Useful for price and volume predictions
-

8. Explained Variance Score

Explanation

Measures the proportion of variance explained by the model compared to total variance.

Mathematical Formula

$$\text{ExplainedVariance} = 1 - (\text{Var}(y - \hat{y}) / \text{Var}(y))$$

When to Use

- Model performance assessment
- Variance explanation
- Prediction quality evaluation

Python Implementation

```
from sklearn.metrics import explained_variance_score

def calculate_explained_variance(y_true, y_pred):
    return explained_variance_score(y_true, y_pred)
```

Pros

- Provides variance explanation
- Sensitive to prediction errors
- Normalized score

Cons

- Similar to R-squared
- Assumes linear relationships

- Limited interpretability

Fun Facts

- Used in signal processing
 - Indicates model's explanatory power
-

9. Mean Squared Logarithmic Error (MSLE)

Explanation

Mean Squared Logarithmic Error (MSLE) is a loss function that applies logarithmic scaling to reduce the impact of large errors while preserving relative differences.

Mathematical Formula

$$MSLE = (1/n) * \sum[(\log(y_i + 1) - \log(\hat{y}_i + 1))^2]$$

When to Use

- When dealing with data with exponential growth
- Useful for metrics where relative errors are more important than absolute errors
- Recommended for scenarios with wide range of target values
- Particularly effective for financial, economic, or scientific data with exponential characteristics

Python Implementation

```
import numpy as np

def mean_squared_log_error(y_true, y_pred):
    return np.mean(np.square(np.log1p(y_true) - np.log1p(y_pred)))
```

Pros

- Reduces impact of large outliers
- Handles wide range of scales effectively
- Emphasizes relative prediction accuracy

Cons

- Not suitable for negative predictions
- Can be sensitive to small changes in log-scaled values
- Less interpretable compared to MSE

Fun Facts

- Logarithmic transformation is similar to log-based normalization
- Often used in competitions like Kaggle for certain prediction tasks
- Closely related to log transformation in statistical modeling

10. Log-Cosh Loss

Explanation

Log-Cosh Loss is a smooth approximation of Mean Absolute Error (MAE) that provides better numerical stability.

Mathematical Formula

$$\text{Log-Cosh Loss} = (1/n) * \sum [\log(\cosh(y_i - \hat{y}_i))]$$

When to Use

- When you want a smooth loss function
- Suitable for regression problems with potential outliers
- Provides a balance between MSE and MAE

Python Implementation

```
import numpy as np

def log_cosh_loss(y_true, y_pred):
    return np.mean(np.log(np.cosh(y_pred - y_true)))
```

Pros

- Smooth and differentiable

- Less sensitive to outliers compared to MSE
- Computationally efficient

Cons

- Can be less interpretable
- Performance depends on specific dataset characteristics
- Might not capture all error nuances

Fun Facts

- Mathematically similar to Huber loss
 - Provides a good compromise between MAE and MSE
 - Commonly used in deep learning optimization
-

11. Quantile Loss

Explanation

Quantile Loss allows prediction of specific quantiles of the target variable, providing more flexible regression modeling.

Mathematical Formula

$$QuantileLoss = (1/n) * \sum [max(q * (y_i - \hat{y}_i), (q - 1) * (y_i - \hat{y}_i))]$$

Where:

- q is the quantile

When to Use

- Predicting specific percentiles of target variable
- Asymmetric prediction scenarios
- Risk assessment and financial modeling
- Capturing uncertainty in predictions

Python Implementation

```
import numpy as np

def quantile_loss(y_true, y_pred, quantile=0.5):
    errors = y_true - y_pred
    return np.mean(np.maximum(quantile * errors, (quantile - 1) * errors))
```

Pros

- Allows flexible probabilistic predictions
- Can model different parts of the distribution
- Useful for risk-aware modeling

Cons

- More complex to interpret
- Computationally more expensive
- Requires careful quantile selection

Fun Facts

- Used in financial risk modeling
- Enables prediction of confidence intervals
- Powerful technique in machine learning uncertainty estimation

12. Relative Absolute Error (RAE)

Explanation

Relative Absolute Error (RAE) measures prediction accuracy relative to a naive baseline prediction.

Mathematical Formula

$$RAE = \sum |y_i - \hat{y}_i| / \sum |y_i - \text{mean}(y)|$$

When to Use

- Comparing different model performances

- Normalizing error across different scales
- Assessing relative prediction quality

Python Implementation

```
import numpy as np

def relative_absolute_error(y_true, y_pred):
    numerator = np.sum(np.abs(y_true - y_pred))
    denominator = np.sum(np.abs(y_true - np.mean(y_true)))
    return numerator / denominator
```

Pros

- Scale-independent metric
- Easy to interpret
- Provides relative performance assessment

Cons

- Sensitive to extreme values
- Can be misleading with small datasets
- Doesn't provide absolute error magnitude

Fun Facts

- Part of the family of relative error metrics
- Used in academic and research model evaluations
- Helps compare models across different domains

13. Relative Squared Error (RSE)

Explanation

Relative Squared Error (RSE) compares squared prediction errors to squared errors of a baseline model.

Mathematical Formula

$$RSE = \frac{\sum[(y_i - \hat{y}_i)^2]}{\sum[(y_i - \text{mean}(y))^2]}$$

When to Use

- Comparing model performances
- Normalizing error across different datasets
- Providing relative error assessment

Python Implementation

```
import numpy as np

def relative_squared_error(y_true, y_pred):
    numerator = np.sum((y_true - y_pred)**2)
    denominator = np.sum((y_true - np.mean(y_true))**2)
    return numerator / denominator
```

Pros

- Provides normalized error metric
- Penalizes larger errors more
- Useful for model comparison

Cons

- Squares errors, amplifying outlier impact
- Less interpretable than absolute metrics
- Can be misleading with small datasets

Fun Facts

- Closely related to R-squared metric
- Common in statistical modeling
- Helps assess model improvement over baseline

14. Symmetric Mean Absolute Percentage Error (SMAPE)

Explanation

SMAPE provides a symmetric percentage error metric that handles both overestimation and underestimation equally.

Mathematical Formula

$$SMAPE = (1/n) * \sum [|y_i - \hat{y}_i| / ((|y_i| + |\hat{y}_i|) / 2)] * 100$$

When to Use

- Time series forecasting
- Comparing models with different scales
- Handling both positive and negative predictions

Python Implementation

```
import numpy as np

def symmetric_mean_absolute_percentage_error(y_true, y_pred):
    return np.mean(np.abs(y_true - y_pred) / ((np.abs(y_true) + np.abs(y_pred)) / 2)) * 100
```

Pros

- Symmetric handling of errors
- Percentage-based, easy to interpret
- Works with both positive and negative values

Cons

- Can be unstable with values close to zero
- Sensitive to small absolute differences
- Might not be suitable for all datasets

Fun Facts

- Recommended by many forecasting competitions
 - More robust than traditional MAPE
 - Widely used in demand forecasting
-

15. Mean Bias Deviation (MBD)

Explanation

Mean Bias Deviation measures the average bias in predictions, indicating systematic over or under-estimation.

Mathematical Formula

$$MBD = (1/n) * \sum(\hat{y}_i - y_i)$$

When to Use

- Checking systematic model biases
- Quality control in predictive models
- Understanding model prediction tendencies

Python Implementation

```
import numpy as np

def mean_bias_deviation(y_true, y_pred):
    return np.mean(y_pred - y_true)
```

Pros

- Simple to calculate
- Directly shows model bias direction
- Helps identify systematic errors

Cons

- Positive and negative errors can cancel out
- Less informative for complex models
- Doesn't capture error magnitude

Fun Facts

- Important in scientific and engineering modeling
- Helps improve model calibration

- Commonly used in climate and environmental prediction

16. Mean Directional Accuracy (MDA)

Explanation

Mean Directional Accuracy measures the model's ability to predict the correct direction of change.

Mathematical Formula

$$MDA = (1/n) * \sum [sign(y_i - y_{i-1}) == sign(\hat{y}_i - \hat{y}_{i-1})]$$

When to Use

- Time series and sequential predictions
- Financial market forecasting
- Trend prediction models

Python Implementation

```
import numpy as np

def mean_directional_accuracy(y_true, y_pred):
    direction_true = np.diff(y_true)
    direction_pred = np.diff(y_pred)
    return np.mean(np.sign(direction_true) == np.sign(direction_pred))
```

Pros

- Focuses on directional prediction
- Useful for trend-based forecasting
- Simple to understand and implement

Cons

- Ignores magnitude of predictions
- Less informative for non-sequential data
- Can be misleading with noisy data

Fun Facts

- Critical in financial and economic modeling
- Used in technical analysis of stock markets
- Complements other accuracy metrics

17. Huber Loss

Explanation

Combines MSE and MAE, being less sensitive to outliers while maintaining quadratic behavior for small errors.

Mathematical Formula

$$HuberLoss = (1/n) * \sum [0.5 * (y_i - \hat{y}_i)^2 \text{ if } |y_i - \hat{y}_i| \leq \delta \text{ else } \delta * (|y_i - \hat{y}_i| - 0.5 * \delta)]$$

Where:

- δ is a threshold parameter

When to Use

- Robust regression
- Outlier-prone datasets
- Machine learning optimization

Python Implementation

```
import numpy as np

def huber_loss(y_true, y_pred, delta=1.0):
    error = y_true - y_pred
    is_small_error = np.abs(error) <= delta
    squared_loss = 0.5 * error**2
    linear_loss = delta * np.abs(error) - 0.5 * delta**2
    return np.mean(np.where(is_small_error, squared_loss, linear_loss))
```

Pros

- Robust to outliers
- Combines quadratic and linear loss
- Smooth transition

Cons

- Requires hyperparameter tuning
- More complex implementation
- Computationally expensive

Fun Facts

- Named after Peter Huber
 - Used in robust statistics
-